

4_10xPBMCom_comparison_geneIntersection

August 31, 2026

```
[1]: library(rtracklayer)
library(ggplot2)
library(Seurat)

getwd()
dir.create("figures_10xPBMCom")
dir.create("data")

colorTools = c("SoloTE"="#A4DD9B", #6fc69d",
               "Stellarscope"="#4f5d93",
               "STARsolo"="#f0df93")

dataset_id <- "10xPBMCom"
```

Loading required package: GenomicRanges

Loading required package: stats4

Loading required package: BiocGenerics

Attaching package: 'BiocGenerics'

The following objects are masked from 'package:stats':

IQR, mad, sd, var, xtabs

The following objects are masked from 'package:base':

anyDuplicated, aperm, append, as.data.frame, basename, cbind,
colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,
match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,
Position, rank, rbind, Reduce, rownames, sapply, setdiff, sort,

table, tapply, union, unique, unsplit, which.max, which.min

Loading required package: S4Vectors

Attaching package: 'S4Vectors'

The following object is masked from 'package:utils':

findMatches

The following objects are masked from 'package:base':

expand.grid, I, unname

Loading required package: IRanges

Loading required package: GenomeInfoDb

Loading required package: SeuratObject

Loading required package: sp

Attaching package: 'sp'

The following object is masked from 'package:IRanges':

%over%

Attaching package: 'SeuratObject'

The following object is masked from 'package:GenomicRanges':

intersect

The following object is masked from 'package:GenomeInfoDb':

intersect

The following object is masked from 'package:IRanges':

```
intersect
```

The following object is masked from 'package:S4Vectors':

```
intersect
```

The following object is masked from 'package:BiocGenerics':

```
intersect
```

The following objects are masked from 'package:base':

```
intersect, t
```

```
"/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets'
```

```
Warning message in dir.create("figures_10xPBM"):

```

```
"'figures_10xPBM' already exists"

```

```
Warning message in dir.create("data"):

```

```
"'data' already exists"

```

```
[2]: # theme_paper <- function(base_size = 17, base_family = "") {
#   theme_minimal(base_size = base_size, base_family = base_family) +
#     theme(
#       plot.title = element_text(face = "bold", size = base_size + 2, hjust = 0.5),
#       axis.title = element_text(size = base_size),
#       axis.text = element_text(size = base_size * 0.9),
#       legend.title = element_text(size = base_size),
#       legend.text = element_text(size = base_size * 0.9),
#       panel.grid.major = element_line(linewidth = 0.4),
#       panel.grid.minor = element_blank(),
#       plot.margin = margin(10, 10, 10, 10)
#     )
# }

# theme_set(theme_paper())
```

```
[3]: #save.image("workspaces/afterCorrelations_wSTAR.RData")
load("workspaces/afterCorrelations_10xPBMc_wSTAR_noPlatelet.RData")
gc()
```

		used	(Mb)	gc trigger	(Mb)	max used	(Mb)
A matrix: 2×6 of type dbl	Ncells	33246026	1775.6	60153791	3212.6	33249724	1775.8
	Vcells	253633988	1935.1	323392462	2467.3	253654538	1935.3

1 Gene Intersections

```
[4]: library(rtracklayer)
# import the annotation used for snakemake
gene_annotation <- rtracklayer::import("/mnt/TEdataStorage_2T/snakemake_data/
  ↪references/hg38/genecode.v30.annotation.gtf.gz")
names(mcols(gene_annotation))
```

1. 'source' 2. 'type' 3. 'score' 4. 'phase' 5. 'gene_id' 6. 'gene_type' 7. 'gene_name' 8. 'level'
 9. 'havana_gene' 10. 'transcript_id' 11. 'transcript_type' 12. 'transcript_name' 13. 'transcript_support_level'
 14. 'tag' 15. 'havana_transcript' 16. 'exon_number' 17. 'exon_id' 18. 'ont'
 19. 'protein_id' 20. 'ccdsid'

```
[5]: gene_annotation_df <- as.data.frame(gene_annotation)

# Extract only exon entries
exons <- gene_annotation[gene_annotation$type == "exon"]
names(exons) <- exons$gene_name
```

```
[6]: # keep only TEs that are in both SoloTE and Stellarscope
annotation_common <- na.omit(conversionTable)

cat("Classes not in SoloTE annotation: ",
    paste(setdiff(unique(conversionTable$class),
  ↪unique(annotation_common$class)), collapse = ", "),
    "\n")
```

Classes not in SoloTE annotation: Satellite, Retroposon, NA, Unknown, RNA

```
[7]: objTElist <- list()

objTElist[["SoloTE"]] <- objTE_soloTE
objTElist[["Stellarscope"]] <- objTE_stellarscope
objTElist[["STARsolo"]] <- objTE_STARsolo

objTElist
```

\$SoloTE

An object of class Seurat

6758 features across 1056 samples within 1 assay

Active assay: RNA (6758 features, 4000 variable features)
 3 layers present: counts, data, scale.data
 2 dimensional reductions calculated: pca, umap

\$Stellarscope

An object of class Seurat
 6367 features across 1056 samples within 1 assay
 Active assay: RNA (6367 features, 4000 variable features)
 3 layers present: counts, data, scale.data
 2 dimensional reductions calculated: pca, umap

\$STARsolo

An object of class Seurat
 4955 features across 1056 samples within 1 assay
 Active assay: RNA (4955 features, 4000 variable features)
 3 layers present: counts, data, scale.data
 2 dimensional reductions calculated: pca, umap

```
[8]: # Compute overlaps to exons and save lists of loci
featuresList <- list()

bedList <- list()
grList <- list()
exonOverlapList <- list()
toolOverlappingGrList <- list()
toolOverlappingList <- list()

for(tool in names(objTElist)){
  if(tool == "SoloTE"){
    bedList[[tool]] <- conversionTable[conversionTable$soloteID %in%
↪Features(objTElist[[tool]]),
    c("chr","start","end","strand","stellarscopeID")]
    print(nrow(bedList[[tool]]))
  }else{
    bedList[[tool]] <- conversionTable[conversionTable$stellarscopeID %in%
↪Features(objTElist[[tool]]),
    c("chr","start","end","strand","stellarscopeID")]
    print(nrow(bedList[[tool]]))
  }

  bedList[[tool]] <- bedList[[tool]][!
↪duplicated(bedList[[tool]]$stellarscopeID),]
  bedList[[tool]] <- bedList[[tool]][!is.na(bedList[[tool]]$stellarscopeID),]
  featuresList[[tool]] <- bedList[[tool]]$stellarscopeID
  print(nrow(bedList[[tool]]))
  rownames(bedList[[tool]]) <- bedList[[tool]]$stellarscopeID
```

```

# turn into GR object
grList[[tool]] <- makeGRangesFromDataFrame(bedList[[tool]])
exonOverlapList[[tool]] <- findOverlaps(grList[[tool]], exons, minoverlap = 10)

toolOverlappingGrList[[tool]] <- grList[[tool]][unique(exonOverlapList[[tool]]@from)]
toolOverlappingList[[tool]] <- rownames(as.data.frame(toolOverlappingGrList[[tool]]))
#print(as.data.frame(toolOverlappingList[[tool]]))
}

```

```

[1] 6750
[1] 6745
[1] 6367
[1] 6367

```

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

```

- in 'x': MU273340.1, GL000251.2, GL000252.2, GL000253.2, GL000254.2,
GL000255.2, GL000258.2, GL339449.2, GL383582.2, GL949753.2, KI270442.1,
KI270831.1, KI270853.1, KI270857.1, KI270897.1, KI270904.1, KI270908.1,
KQ031389.1, KQ458383.1, KZ559110.1

```

```

- in 'y': chrM

```

Make sure to always combine/compare objects based on the same reference genome (use suppressWarnings() to suppress this warning)."

```

[1] 4955
[1] 4955

```

Warning message in .merge_two_Seqinfo_objects(x, y):

"Each of the 2 combined objects has sequence levels not in the other:

```

- in 'x': MU273330.1, MU273339.1, MU273340.1, MU273397.1, GL000250.2,
GL000251.2, GL000252.2, GL000253.2, GL000254.2, GL000255.2, GL000256.2,
GL000258.2, GL339449.2, GL383518.1, GL383520.2, GL383572.1, GL383575.2,
GL383582.2, GL877876.1, GL949746.1, GL949747.2, GL949748.2, GL949749.2,
GL949750.2, GL949751.2, GL949752.1, GL949753.2, KI270442.1, KI270711.1,
KI270765.1, KI270778.1, KI270801.1, KI270809.1, KI270830.1, KI270831.1,
KI270832.1, KI270844.1, KI270849.1, KI270851.1, KI270853.1, KI270855.1,
KI270857.1, KI270859.1, KI270861.1, KI270862.1, KI270868.1, KI270879.1,
KI270880.1, KI270897.1, KI270903.1, KI270904.1, KI270905.1, KI270908.1,
KI270911.1, KI270926.1, KI270928.1, KI270938.1, KN196485.1, KN196486.1,
KQ031389.1, KQ458383.1, KQ458387.1, KQ458388.1, KQ759761.1, KQ983256.1,
KV575243.1, KZ559114.1

```

```

- in 'y': chrM

```

Make sure to always combine/compare objects based on the same reference genome (use suppressWarnings() to suppress this warning)."

```
[9]: colorTools
```

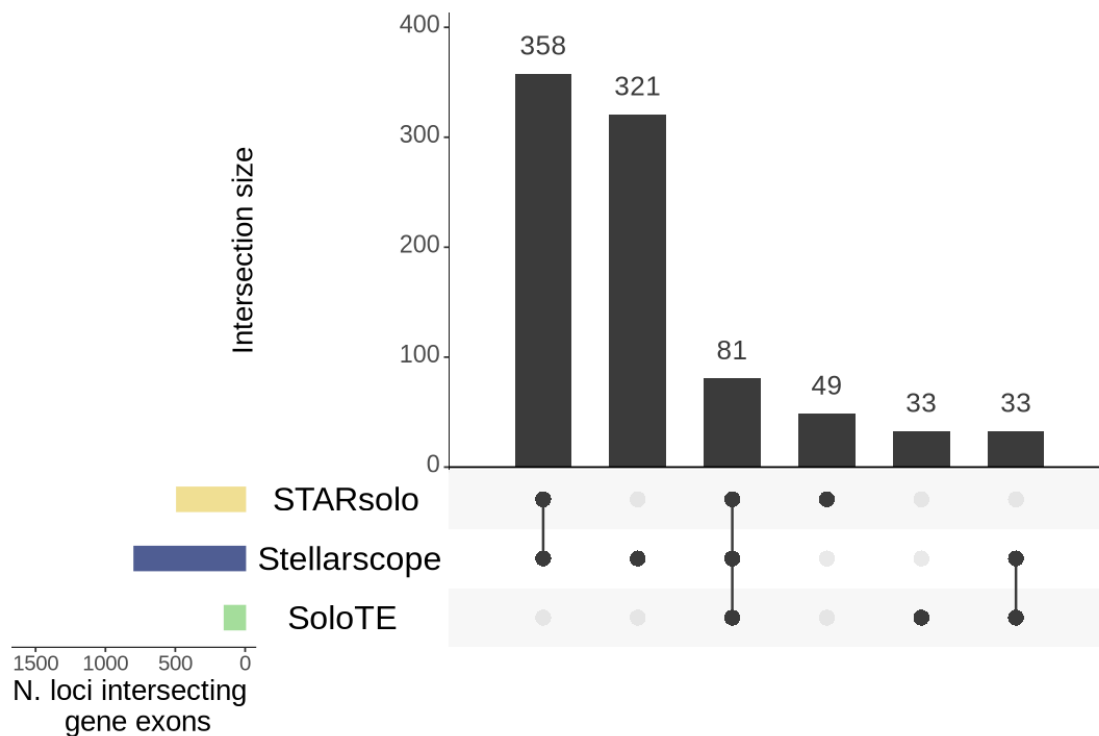
SoloTE '#A4DD9B' Stellerscope '#4f5d93' STARsolo '#f0df93'

```
[13]: library(UpSetR)
options(repr.plot.width=9, repr.plot.height=6)

ph <- UpSetR::upset(fromList(toolOverlappingList),
  sets=names(toolOverlappingList), sets.bar.color=colorTools,
  keep.order = TRUE,
  text.scale = c(2, 2, 2, 1.75, 2.75, 2.5),
  order.by=c("freq"),
  point.size=4, mb.ratio = c(0.63, 0.37),
  set_size.show=F, set_size.scale_max= max(lengths(toolOverlappingList))+800,
  sets.x.label="N. loci intersecting \n gene exons",
  mainbar.y.label="Intersection size")

pdf(paste0("figures_", dataset_id, "/NlociIntersectingGeneExons_upset.pdf"),
  width=9, height=6)
ph
dev.off()
ph
```

pdf: 2



```
[11]: df <- NULL

for(tool in names(objTElist)){

  loci <- bedList[[tool]]$stellarscopeID
  lociOverlappingGenes <- toolOverlappingList[[tool]]

  df <- rbind(df, c(tool, length(loci), length(lociOverlappingGenes)))

}

df <- as.data.frame(df)
df
```

	V1	V2	V3
	<chr>	<chr>	<chr>
A data.frame: 3 × 3	SoloTE	6745	147
	Stellarscope	6367	793
	STARsolo	4955	488

```
[16]: library(dplyr)
library(purrr)
library(tidyr)

make_membership_table <- function(all_lists, gene_lists=NULL) {

  # All loci that appear in at least one tool
  all_loci <- unique(unlist(all_lists))

  # Presence/absence table (automatic)
  df <- tibble(locus = all_loci)

  for (nm in names(all_lists)) {
    df[[nm]] <- df$locus %in% all_lists[[nm]]
  }

  # Optional: add gene-overlap column
  if (!is.null(gene_lists)) {
    gene_loci <- unique(unlist(gene_lists))
    df$gene_overlap <- df$locus %in% gene_loci
  }

  df
}

df <- make_membership_table(featuresList, toolOverlappingList)
head(df)
```


Attaching package: 'dplyr'

The following objects are masked from 'package:GenomicRanges':

intersect, setdiff, union

The following object is masked from 'package:GenomeInfoDb':

intersect

The following objects are masked from 'package:IRanges':

collapse, desc, intersect, setdiff, slice, union

The following objects are masked from 'package:S4Vectors':

first, intersect, rename, setdiff, setequal, union

The following objects are masked from 'package:BiocGenerics':

combine, intersect, setdiff, union

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'purrr'

The following object is masked from 'package:GenomicRanges':

reduce

The following object is masked from ‘package:IRanges’:

reduce

Attaching package: ‘tidyr’

The following object is masked from ‘package:S4Vectors’:

expand

A tibble: 6 × 5

locus <chr>	SoloTE <lgl>	Stellarscope <lgl>	STARsolo <lgl>	gene_overlap <lgl>
MT2-Mm-dup2	TRUE	TRUE	TRUE	FALSE
MLT1A-dup2	TRUE	TRUE	TRUE	FALSE
L1-Mus4-dup3	TRUE	TRUE	TRUE	FALSE
MT2-Mm-dup3	TRUE	TRUE	TRUE	FALSE
L1Md-F2-dup62	TRUE	TRUE	TRUE	FALSE
L1MC2-dup3	TRUE	TRUE	FALSE	FALSE

```
[17]: df <- df %>%
  mutate(
    group = pmap_chr(select(., SoloTE:STARsolo), ~ {
      here <- c(...)
      tools <- names(here)[here]
      if (length(tools) == 0) return(NA_character_)
      paste(tools, collapse = "&")
    })
  )
```

```
[18]: summary_df <- df %>%
  group_by(group) %>%
  summarise(
    N = n(),
    gene_N = sum(gene_overlap),
    percent_gene = round(100 * gene_N / N, 1),
    .groups = "drop"
  ) %>%
  arrange(desc(N))
```

```
[19]: lengths(toolOverlappingList)
summary_df
```

SoloTE	588 Stellarscope	3305 STARsolo			1474
	group <chr>	N <int>	gene_N <int>	percent_gene <dbl>	
A tibble: 7 × 4	STARsolo	21035	287	1.4	
	SoloTE	19726	211	1.1	
	SoloTE&Stellarscope	17215	179	1.0	
	SoloTE&Stellarscope&STARsolo	10630	196	1.8	
	Stellarscope&STARsolo	6167	989	16.0	
	Stellarscope	4701	1941	41.3	
	SoloTE&STARsolo	97	2	2.1	

```
[20]: mix_colors <- function(cols) {
  cols_rgb <- t(col2rgb(cols))
  avg_rgb <- colMeans(cols_rgb)
  rgb(avg_rgb[1], avg_rgb[2], avg_rgb[3], maxColorValue = 255)
}

library(purrr)

# 1-way (original tool colors)
color_1way <- colorTools

# 2-way mixes
color_2way <- list(
  "SoloTE&Stellarscope" = mix_colors(colorTools[c("SoloTE", "Stellarscope")]),
  "SoloTE&STARsolo"     = mix_colors(colorTools[c("SoloTE", "STARsolo")]),
  "Stellarscope&STARsolo" = mix_colors(colorTools[c("Stellarscope", "STARsolo")])
)

# 3-way mix
color_3way <- c(
  "SoloTE&Stellarscope&STARsolo" =
    mix_colors(colorTools[c("SoloTE", "Stellarscope", "STARsolo")])
)

# Combine everything
intersection_colors <- c(color_1way, color_2way, color_3way)
intersection_colors
```

```
$SoloTE '#A4DD9B'
$Stellarscope '#4f5d93'
$STARsolo '#f0df93'
$'SoloTE&Stellarscope' '#799D97'
$'SoloTE&STARsolo' '#CADE97'
$'Stellarscope&STARsolo' '#9F9E93'
```

\$'SoloTE&Stellarscope&STARsolo' '#A1B395'

[21]: summary_df

	group <chr>	N <int>	gene_N <int>	percent_gene <dbl>
A tibble: 7 × 4	STARsolo	21035	287	1.4
	SoloTE	19726	211	1.1
	SoloTE&Stellarscope	17215	179	1.0
	SoloTE&Stellarscope&STARsolo	10630	196	1.8
	Stellarscope&STARsolo	6167	989	16.0
	Stellarscope	4701	1941	41.3
	SoloTE&STARsolo	97	2	2.1

```
[48]: options(repr.plot.width=9, repr.plot.height=4)
library(ggpubr)

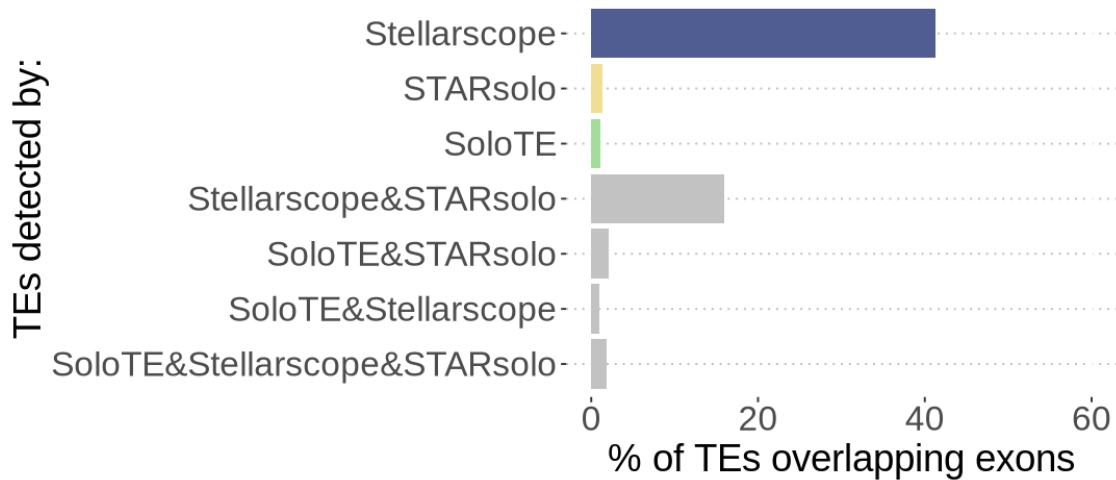
df <- summary_df
# Step 1: compute size
df$size <- sapply(df$group, function(x) length(strsplit(x, "&")[[1]]))

# Step 2: get **unique groups** and sort by size
unique_groups <- unique(df$group)
group_levels <- unique_groups[order(sapply(unique_groups, function(x)
  ↪length(strsplit(x, "&")[[1]])))]

# descending within each size
df <- df %>%
  arrange(-size, percent_gene) # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group) # preserve new order

ggplot(df, aes(y = group, x = percent_gene, fill = group)) +
  geom_col(alpha=1) +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("% of TEs overlapping exons") +
  ylab("TEs detected by:") +
  xlim(0,60) +
  theme_pubclean() +
  theme(text=element_text(size=22),
        axis.text=element_text(size=20),
        legend.position = "none"
  )
ggsave(paste0("figures_", dataset_id, "/percTEsOverlappingExons.pdf"), width=8,
  ↪height=6)
```



```
[51]: options(repr.plot.width=9, repr.plot.height=4)

df <- summary_df
# Step 1: compute size
df$size <- sapply(df$group, function(x) length(strsplit(x, "&")[[1]]))

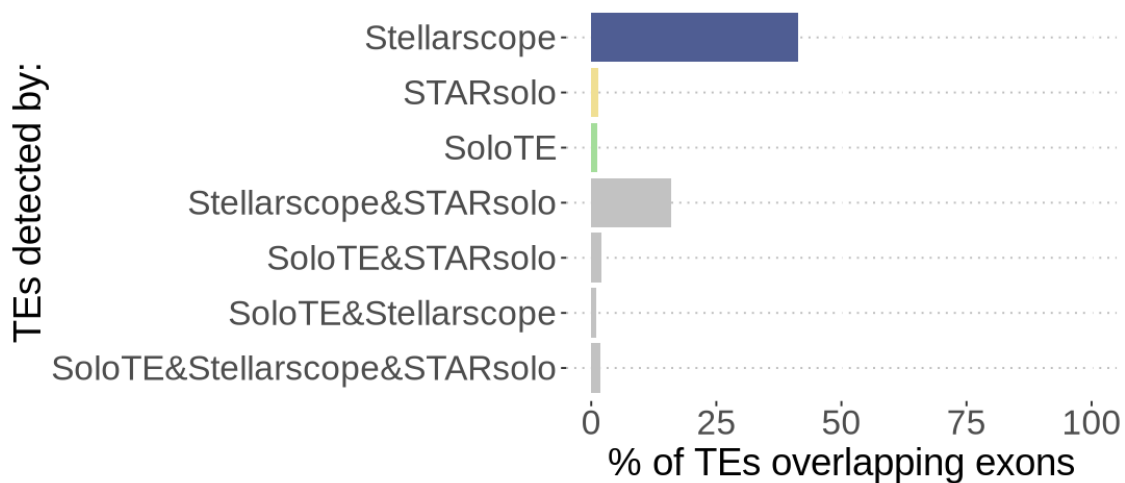
# Step 2: get **unique groups** and sort by size
unique_groups <- unique(df$group)
group_levels <- unique_groups[order(sapply(unique_groups, function(x)
  ↪length(strsplit(x, "&")[[1]])))]

# descending within each size
df <- df %>%
  arrange(-size, percent_gene) # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group) # preserve new order

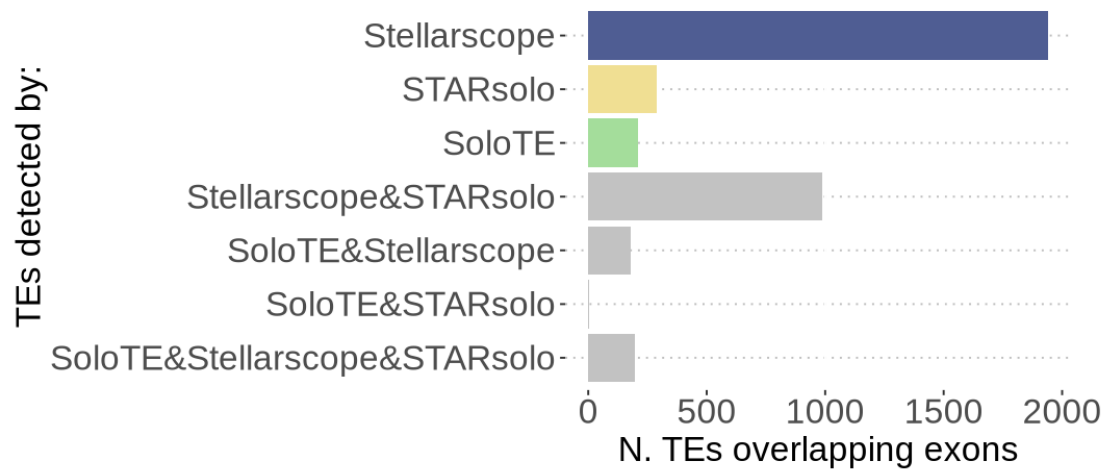
ggplot(df, aes(y = group, x = percent_gene, fill = group)) +
  geom_col() +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("% of TEs overlapping exons") +
  ylab("TEs detected by:") +
  xlim(0,100) +
  theme_pubclean() +
  theme(text=element_text(size=22),
        axis.text=element_text(size=20),
        legend.position = "none"
  )
```

```
ggsave(paste0("figures_", dataset_id, "/percTEsOverlappingExons_max100.pdf"),
  width=8, height=4)
```



```
[50]: # descending within each size
df <- df %>%
  arrange(-size, gene_N) # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group) # preserve new order
ggplot(df, aes(y = group, x = gene_N, fill = group)) +
  geom_col() +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("N. TEs overlapping exons") +
  ylab("TEs detected by:") +
  theme_pubclean() +
  theme(text=element_text(size=20),
        axis.text=element_text(size=20),
        legend.position = "none",
        plot.margin = margin(t = 10, r = 30, b = 10, l = 10) # top, right,
  bottom, left
  )
ggsave(paste0("figures_", dataset_id, "/nTEsOverlappingExons.pdf"), width=8,
  height=4)
```



[]: